

# UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

---

## Computación Síncrona y Asíncrona en Sistemas Distribuidos

---

|                     |                                          |
|---------------------|------------------------------------------|
| <b>Asignación:</b>  | GA-SUM-01 — EXP-1 — Exposición 1er Corte |
| <b>Materia:</b>     | Aplicaciones Distribuidas                |
| <b>Docente:</b>     | Ing. Guerrero Ulloa Gleiston Cicerón     |
| <b>Nivel:</b>       | 7mo Nivel                                |
| <b>Año Lectivo:</b> | 2026 – 2027 PPA                          |

---

### Integrantes del Grupo

Carpio Mendoza Carlos José

Farinango Guandinango Freddy Vladimir

Vinueza Sánchez Harold Nicolás

---

Quevedo, Los Ríos — Ecuador

May 27, 2026

## Contents

---

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Resumen</b>                                             | <b>3</b>  |
| <b>2</b> | <b>Introducción</b>                                        | <b>3</b>  |
| <b>3</b> | <b>Computación Síncrona</b>                                | <b>3</b>  |
| 3.1      | Definición y características fundamentales . . . . .       | 3         |
| 3.2      | Modelo de comunicación síncrona . . . . .                  | 4         |
| 3.3      | Ejemplos del modelo síncrono . . . . .                     | 4         |
| 3.4      | Tecnologías de comunicación síncrona . . . . .             | 5         |
| 3.4.1    | REST (Representational State Transfer) . . . . .           | 5         |
| 3.4.2    | gRPC (Google Remote Procedure Call) . . . . .              | 5         |
| 3.4.3    | HTTP Blocking . . . . .                                    | 5         |
| 3.5      | Casos de uso reales del modelo síncrono . . . . .          | 5         |
| 3.6      | Problemas del modelo síncrono . . . . .                    | 6         |
| <b>4</b> | <b>Computación Asíncrona</b>                               | <b>6</b>  |
| 4.1      | Definición y características fundamentales . . . . .       | 6         |
| 4.2      | Modelo de comunicación asíncrona . . . . .                 | 7         |
| 4.3      | Ejemplos del modelo asíncrono . . . . .                    | 7         |
| 4.4      | Características clave del modelo asíncrono . . . . .       | 7         |
| 4.5      | Casos de uso reales del modelo asíncrono . . . . .         | 8         |
| <b>5</b> | <b>Comparativa: Síncrono vs. Asíncrono</b>                 | <b>8</b>  |
| 5.1      | ¿Cuándo usar cada modelo? . . . . .                        | 9         |
| <b>6</b> | <b>El Contexto Distribuido</b>                             | <b>10</b> |
| 6.1      | Sistemas distribuidos y comunicación entre nodos . . . . . | 10        |
| 6.2      | Comunicación distribuida síncrona . . . . .                | 10        |
| 6.3      | Comunicación distribuida asíncrona . . . . .               | 10        |
| <b>7</b> | <b>Middleware y Message Broker</b>                         | <b>11</b> |
| 7.1      | Definición de Middleware . . . . .                         | 11        |
| 7.2      | Definición de Message Broker . . . . .                     | 11        |
| 7.3      | Tecnologías comunes de Message Broker . . . . .            | 11        |
| <b>8</b> | <b>Arquitectura Basada en Eventos (EDA)</b>                | <b>12</b> |
| 8.1      | Concepto y fundamentos . . . . .                           | 12        |
| 8.2      | Componentes de la EDA . . . . .                            | 12        |
| 8.3      | Ventajas de la EDA . . . . .                               | 12        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Caso de Estudio: El Modelo Bancario Híbrido</b>           | <b>13</b> |
| 9.1       | Introducción al caso . . . . .                               | 13        |
| 9.2       | Fase síncrona: consistencia crítica . . . . .                | 13        |
| 9.3       | Fase asíncrona: operaciones no bloqueantes . . . . .         | 13        |
| 9.4       | Tecnologías utilizadas en la arquitectura bancaria . . . . . | 14        |
| <b>10</b> | <b>Tecnologías de Comunicación en Detalle</b>                | <b>14</b> |
| 10.1      | Tecnologías síncronas . . . . .                              | 14        |
| 10.1.1    | RPC (Remote Procedure Call) . . . . .                        | 14        |
| 10.1.2    | REST (Representational State Transfer) . . . . .             | 14        |
| 10.1.3    | HTTP Blocking . . . . .                                      | 15        |
| 10.2      | Tecnologías asíncronas . . . . .                             | 15        |
| 10.2.1    | Apache Kafka . . . . .                                       | 15        |
| 10.2.2    | RabbitMQ . . . . .                                           | 15        |
| <b>11</b> | <b>Discusión y Análisis Crítico</b>                          | <b>15</b> |
| 11.1      | Impacto en el rendimiento . . . . .                          | 15        |
| 11.2      | El dilema de la consistencia . . . . .                       | 16        |
| 11.3      | Escalabilidad y resiliencia . . . . .                        | 16        |
| 11.4      | Tendencias actuales . . . . .                                | 16        |
| <b>12</b> | <b>Conclusiones</b>                                          | <b>16</b> |
|           | <b>Referencias</b>                                           | <b>18</b> |

## Resumen

---

El presente informe expone los fundamentos teóricos y prácticos de los dos paradigmas dominantes de comunicación en los sistemas distribuidos modernos: la *computación síncrona* y la *computación asíncrona*. Se analiza en detalle qué diferencia a cada modelo, cuáles son sus ventajas, limitaciones y los escenarios en los que resulta más conveniente emplear cada uno. Se estudian también las tecnologías de soporte —REST, gRPC, Apache Kafka, RabbitMQ, entre otras— y se examina un caso de estudio bancario que ilustra cómo ambos paradigmas conviven en arquitecturas híbridas de producción. Todas las afirmaciones se sustentan en referencias académicas con identificador DOI verificable, indexadas en bases de datos reconocidas de ingeniería.

**Palabras clave:** computación síncrona, computación asíncrona, sistemas distribuidos, arquitectura basada en eventos, middleware, message broker, escalabilidad.

## Introducción

---

Los sistemas informáticos modernos raramente operan como unidades aisladas; por el contrario, se componen de múltiples servicios, nodos y componentes que deben comunicarse de manera eficiente para ofrecer una experiencia coherente al usuario final. En este contexto, la elección del modelo de comunicación resulta fundamental para determinar el rendimiento, la resiliencia y la escalabilidad de una aplicación [1].

Históricamente, la arquitectura cliente-servidor basada en protocolos síncronos —como HTTP convencional— fue la solución predominante para construir servicios de red. Sin embargo, con la explosión de la computación en la nube, los microservicios y los sistemas de alta disponibilidad, la comunicación asíncrona emergió como un complemento indispensable que permite desacoplar componentes, gestionar picos de carga y mejorar la tolerancia a fallos [2].

Este informe parte de los contenidos abordados en la exposición *Computación Síncrona y Asíncrona* y los amplía con fundamento académico, con el objetivo de proporcionar un análisis profundo y verificable de ambos paradigmas, sus tecnologías asociadas y sus aplicaciones en entornos reales.

## Computación Síncrona

---

### Definición y características fundamentales

La **computación síncrona** es el modelo de comunicación en el que un proceso emisor envía una solicitud y permanece bloqueado —es decir, suspende su propia ejecución—

hasta recibir la respuesta del proceso receptor. Este comportamiento implica un *acoplamiento temporal* fuerte entre las partes: el cliente y el servidor deben estar simultáneamente disponibles para que la interacción se lleve a cabo [3].

Las características principales de este modelo son las siguientes:

- **Secuencialidad estricta:** las tareas se ejecutan en un orden determinístico; ninguna operación siguiente puede iniciarse hasta que la anterior haya concluido.
- **Bloqueo del hilo de ejecución:** el proceso queda inactivo mientras aguarda la respuesta, lo que representa un consumo improductivo de recursos del sistema.
- **Consistencia inmediata:** al recibir la respuesta en tiempo real, el cliente tiene garantía de que la información está actualizada en el momento de la consulta.
- **Acoplamiento temporal fuerte:** ambos extremos de la comunicación deben estar activos simultáneamente, lo que incrementa la dependencia entre componentes [4].

## Modelo de comunicación síncrona

El flujo de una interacción síncrona puede resumirse en tres fases:

1. **El cliente solicita:** inicia la operación y entra en estado de espera activa (*blocking wait*).
2. **El servidor procesa:** ejecuta la lógica de negocio requerida.
3. **Respuesta inmediata:** el servidor devuelve el resultado y el cliente reanuda su ejecución.

Este modelo es análogo, en el mundo real, a una llamada telefónica convencional: ambas partes deben estar presentes al mismo tiempo para que el intercambio de información ocurra. Si una de ellas no está disponible, la comunicación no puede realizarse.

## Ejemplos del modelo síncrono

**Llamada telefónica:** requiere la presencia simultánea de ambos interlocutores para intercambiar información en tiempo real.

**Petición HTTP clásica:** el navegador envía una solicitud y permanece en espera («cargando») hasta que el servidor devuelve el contenido HTML o los datos JSON.

**Comunicación cliente-servidor:** el protocolo HTTP estándar opera según el esquema Request → Wait → Response.

## Tecnologías de comunicación síncrona

### *REST (Representational State Transfer)*

REST es un estilo arquitectónico para el diseño de APIs web basado en el protocolo HTTP. Sus operaciones son *stateless* —sin estado— y los recursos se identifican mediante URLs uniformes. Es la tecnología de comunicación síncrona más extendida en la industria gracias a su simplicidad y la amplia disponibilidad de herramientas de soporte [5].

### *gRPC (Google Remote Procedure Call)*

gRPC es un framework de código abierto para la invocación remota de procedimientos que utiliza HTTP/2 como protocolo de transporte y *Protocol Buffers* como formato de serialización binaria. Ofrece alto rendimiento, soporte multidioma e interoperabilidad en entornos de microservicios [6]. Al igual que REST, el modelo de invocación por defecto es síncrono: el cliente espera la respuesta antes de continuar.

### *HTTP Blocking*

El HTTP convencional (*HTTP/1.1*) implementa por defecto un modelo de bloqueo en el que el cliente abre una conexión TCP, envía la solicitud y suspende su ejecución hasta obtener una respuesta completa. Este comportamiento, si bien simple de implementar, puede generar cuellos de botella bajo carga concurrente elevada [7].

## Casos de uso reales del modelo síncrono

Los escenarios en los que la comunicación síncrona es la opción preferida son aquellos en que se requiere una **confirmación inmediata** del resultado de una operación:

- **Login bancario:** la validación de credenciales debe producir una respuesta instantánea para garantizar acceso seguro.
- **Consulta de saldo:** el usuario necesita ver el balance actualizado al momento exacto de la petición.
- **Compra en línea:** el proceso de checkout requiere confirmación inmediata de la transacción y el estado del pedido.
- **Pago con tarjeta:** la autorización de pagos debe completarse en segundos tanto en comercios físicos como digitales.

## Problemas del modelo síncrono

A pesar de su simplicidad, el modelo síncrono presenta limitaciones significativas que deben considerarse al diseñar sistemas de alta disponibilidad [8]:

### Problemas del Modelo Síncrono

**Bloqueo del cliente:** el proceso queda inactivo mientras espera, desperdiciando recursos computacionales y limitando la concurrencia efectiva del sistema.

**Timeout:** si el servidor no responde dentro del tiempo límite configurado, la conexión se interrumpe abruptamente, generando errores y degradando la experiencia del usuario.

**Dependencia del servidor:** la disponibilidad del sistema cliente depende directamente de la disponibilidad y rendimiento del servidor; cualquier latencia se propaga de forma inmediata.

**Baja tolerancia a fallos:** la caída de un único servicio en una cadena de llamadas síncronas puede provocar un *efecto cascada* (*cascading failure*) que afecte a múltiples componentes de la aplicación.

## Computación Asíncrona

### Definición y características fundamentales

La **computación asíncrona** es el paradigma en el que el proceso emisor envía un mensaje o solicitud sin quedar bloqueado a la espera de una respuesta; puede continuar ejecutando otras tareas de forma concurrente mientras el proceso receptor atiende la solicitud en algún momento posterior [9].

Las características esenciales de este modelo son:

- **No bloqueo:** el proceso emisor no suspende su ejecución; puede atender otras peticiones mientras espera el resultado.
- **Desacoplamiento temporal:** emisor y receptor no necesitan estar activos al mismo tiempo; los mensajes persisten hasta ser procesados.
- **Respuesta diferida:** la respuesta se gestiona mediante un mecanismo de notificación: un *callback*, un evento o un mensaje en una cola.
- **Alta escalabilidad:** al no bloquear hilos de ejecución, el sistema puede manejar un volumen mucho mayor de operaciones concurrentes [10].

## Modelo de comunicación asíncrona

El flujo típico de una interacción asíncrona involucra tres actores:

1. **El cliente envía:** publica un mensaje en una cola o tópico sin esperar respuesta inmediata y continúa su ejecución.
2. **Cola / Middleware:** el mensaje queda almacenado de forma persistente hasta que un consumidor esté disponible para procesarlo.
3. **El consumidor procesa:** cuando tiene capacidad, extrae el mensaje, ejecuta la lógica correspondiente y, opcionalmente, publica una respuesta en otra cola.

Este modelo se asemeja, en el mundo real, al correo electrónico: el remitente envía el mensaje y continúa con sus actividades; el destinatario lo leerá y responderá cuando lo considere oportuno.

## Ejemplos del modelo asíncrono

**WhatsApp / Chat:** el usuario envía un mensaje y puede cerrar la aplicación o continuar otras conversaciones sin necesidad de que el destinatario esté conectado en ese instante.

**Correo electrónico:** paradigma clásico de comunicación diferida; el receptor responde en función de su propia disponibilidad.

**Colas de mensajes:** sistemas como RabbitMQ permiten que un servicio deposite una tarea en la cola para que otro la procese más adelante, sin acoplamiento temporal entre ambos [11].

## Características clave del modelo asíncrono

**Respuesta diferida:** el cliente no bloquea su hilo de ejecución esperando confirmación inmediata, lo que libera recursos para atender otras peticiones concurrentes.

**Mensajes almacenados:** los mensajes persisten en la cola, lo que garantiza la entrega incluso si el consumidor no está disponible en el momento del envío (tolerancia a fallos).

**Alta escalabilidad:** la arquitectura puede manejar grandes volúmenes de solicitudes y absorber picos de tráfico distribuyendo la carga a lo largo del tiempo.

**Desacoplamiento:** emisor y receptor operan de forma independiente, lo que mejora la resiliencia general del sistema y simplifica el escalado horizontal de cada componente [12].



## Casos de uso reales del modelo asíncrono

El modelo asíncrono es la opción preferida en escenarios de alta carga donde no se requiere confirmación instantánea [13]:

- **Envío de correos electrónicos:** los emails se procesan en segundo plano sin bloquear la respuesta al usuario.
- **Procesamiento de imágenes:** las operaciones de edición o compresión se encolan y ejecutan de forma diferida.
- **Notificaciones masivas:** el envío de alertas a miles de usuarios se distribuye a través de colas de mensajes para evitar sobrecargar el sistema.
- **Streaming de datos:** el análisis continuo de grandes volúmenes de información en tiempo real (IoT, telemetría, logs) se apoya en plataformas asíncronas.
- **Plataformas de transporte (Uber):** la conexión eficiente entre conductores y pasajeros se gestiona mediante eventos asíncronos.
- **Plataformas de contenido (TikTok):** la entrega de contenido personalizado a escala global requiere arquitecturas de procesamiento asíncrono para gestionar millones de interacciones por segundo.

## Comparativa: Síncrono vs. Asíncrono

---

La siguiente tabla resume las diferencias fundamentales entre ambos paradigmas:

Table 1: Comparativa de los modelos de computación síncrono y asíncrono

| Característica           | Computación Síncrona      | Computación Asíncrona       |
|--------------------------|---------------------------|-----------------------------|
| Espera de respuesta      | Sí (bloquea el hilo)      | No (continúa trabajando)    |
| Bloquea procesos         | Sí                        | No                          |
| Velocidad percibida      | Menor (sujeto a latencia) | Mayor (mayor rendimiento)   |
| Complejidad              | Más simple de implementar | Más compleja de gestionar   |
| Escalabilidad            | Limitada                  | Alta                        |
| Acoplamiento             | Fuerte (temporal)         | Débil (desacoplado)         |
| Tolerancia a fallos      | Baja                      | Alta                        |
| Comunicación             | Directa                   | A través de colas o eventos |
| Dependencia del servidor | Alta                      | Baja                        |
| Tecnologías típicas      | REST, gRPC, HTTP Blocking | Kafka, RabbitMQ, ActiveMQ   |

### ¿Cuándo usar cada modelo?

La elección entre un modelo síncrono y uno asíncrono depende de los requisitos funcionales y no funcionales de cada caso [14]:

#### Modelo Síncrono: cuándo usarlo

Úsese cuando se requiera una **confirmación inmediata** del resultado de la operación: autenticación de usuarios, transacciones financieras, consultas de saldo, compras en línea o cualquier escenario en que el cliente no pueda continuar sin conocer el resultado de la petición.

#### Modelo Asíncrono: cuándo usarlo

Úsese cuando el sistema deba manejar **alta carga concurrente** o cuando el resultado de la operación no sea necesario de inmediato: notificaciones masivas, procesamiento por lotes, pipelines de datos, arquitecturas de microservicios

escalables o cualquier tarea de larga duración que pueda ejecutarse en segundo plano.

## El Contexto Distribuido

### Sistemas distribuidos y comunicación entre nodos

Un **sistema distribuido** es un conjunto de nodos de cómputo autónomos, interconectados mediante una red, que colaboran para alcanzar un objetivo común y son percibidos por el usuario final como un sistema único y coherente [15]. La comunicación entre estos nodos puede realizarse mediante cualquiera de los dos paradigmas analizados, dependiendo de los requisitos de consistencia, rendimiento y disponibilidad.

En la práctica, los sistemas distribuidos modernos —incluyendo microservicios, arquitecturas orientadas a eventos y plataformas de streaming— utilizan una combinación estratégica de comunicación síncrona y asíncrona para optimizar cada aspecto de su funcionamiento [16].

### Comunicación distribuida síncrona

En el contexto distribuido, la comunicación síncrona implica que el nodo cliente aguarda activamente la respuesta del nodo servidor. Este modelo presenta las siguientes implicaciones operativas [7]:

- Si el servidor falla, se producen bloqueos, timeouts o esperas indefinidas en el cliente.
- El acoplamiento directo simplifica el diseño del sistema, pero reduce su resiliencia frente a fallos parciales.
- La salud global del sistema depende críticamente de la disponibilidad de cada servicio interconectado en la cadena de llamadas.

### Comunicación distribuida asíncrona

La comunicación distribuida asíncrona introduce un **intermediario** —el *message broker* o cola de mensajes— que desacopla al emisor del receptor. Esto permite que cada nodo opere de forma independiente y que el sistema absorba fallos parciales sin propagarlos a toda la red [17].

## Middleware y Message Broker

---

### Definición de Middleware

El **middleware** es una capa de software que actúa como intermediario entre distintas aplicaciones y servicios, facilitando la comunicación, la gestión de datos y la integración en entornos distribuidos heterogéneos. Va más allá de las funcionalidades del sistema operativo y la red, ofreciendo servicios avanzados como gestión de transacciones, directorios, seguridad y control de acceso [18].

### Definición de Message Broker

Un **message broker** es un tipo específico de middleware orientado a la gestión del envío y la recepción de mensajes entre servicios. Su función principal es desacoplar a los emisores de los receptores, almacenando, enrutando y, en algunos casos, transformando los mensajes para garantizar una comunicación fiable, persistente y asíncrona [11].

### Tecnologías comunes de Message Broker

**Apache Kafka:** plataforma de streaming distribuido de alto rendimiento, optimizada para el procesamiento de eventos en tiempo real y la ingesta de grandes volúmenes de datos. Ofrece durabilidad mediante la replicación de particiones y soporte para múltiples consumidores simultáneos [19].

**RabbitMQ:** broker de mensajes de propósito general basado en el protocolo AMQP. Prioriza la entrega garantizada, la flexibilidad de enrutamiento y la simplicidad de configuración. Es especialmente adecuado para escenarios donde el orden de entrega y la confirmación de procesamiento son críticos [20].

**ActiveMQ:** implementación de código abierto del estándar JMS (*Java Message Service*). Ampliamente utilizado en entornos empresariales Java para la integración de aplicaciones heterogéneas.

**Google Cloud Pub/Sub:** servicio gestionado de mensajería en la nube de Google, basado en el modelo publicador-suscriptor, diseñado para escalar de forma elástica ante picos de tráfico.

**Azure Service Bus:** servicio de mensajería empresarial de Microsoft Azure, con soporte para colas de mensajes y tópicos de publicación-suscripción, con garantías de entrega configurable.

## Arquitectura Basada en Eventos (EDA)

---

### Concepto y fundamentos

La **Arquitectura Orientada a Eventos** (*Event-Driven Architecture*, EDA) es un patrón de diseño de sistemas distribuidos que promueve la comunicación asíncrona y el desacoplamiento entre componentes mediante la producción, detección, consumo y reacción a *eventos* [21].

Un **evento** es la representación de un cambio de estado significativo en el sistema —por ejemplo, un pedido realizado, un usuario registrado o un pago aprobado—. Los eventos son inmutables y no tienen un destino específico; simplemente son publicados y consumidos por cualquier componente que esté suscrito.

### Componentes de la EDA

**Eventos:** registros inmutables de hechos ocurridos en el sistema. No contienen lógica de negocio ni instrucciones de enrutamiento; solo describen lo que ocurrió.

**Message Queues (Colas de mensajes):** almacenan y entregan mensajes de forma confiable, desacoplando a los productores de los consumidores y permitiendo el procesamiento asíncrono y la gestión de picos de carga.

**Pub/Sub (*Publicar/Suscribir*):** los productores publican mensajes en tópicos (*topics*) y los consumidores se suscriben a los tópicos de su interés. Este modelo ofrece mayor flexibilidad que las colas punto a punto, ya que un mismo mensaje puede ser consumido por múltiples suscriptores independientes [22].

### Ventajas de la EDA

Los estudios empíricos demuestran que la adopción de EDA produce mejoras medibles en el rendimiento de los sistemas distribuidos [21]:

- Reducción de la latencia de respuesta en un 19,18% respecto a arquitecturas API-Driven síncronas.
- Reducción de la tasa de errores en un 34,40%.
- Mayor escalabilidad horizontal al no requerir acoplamiento directo entre servicios.
- Mayor tolerancia a fallos gracias a la persistencia de mensajes en las colas.

## Caso de Estudio: El Modelo Bancario Híbrido

---

### Introducción al caso

La industria bancaria constituye un ejemplo paradigmático de cómo los sistemas de producción a escala real combinan ambos paradigmas de comunicación para satisfacer requisitos contrapuestos: por un lado, la necesidad de **consistencia inmediata** en las operaciones financieras; por otro, la necesidad de **escalabilidad y eficiencia** en las operaciones secundarias [14].

Un banco no puede elegir únicamente un paradigma; debe implementar una **arquitectura híbrida** que garantice simultáneamente la seguridad del dinero y la velocidad de la plataforma.

### Fase síncrona: consistencia crítica

Las operaciones que afectan directamente al saldo del cliente se procesan de forma síncrona y bloqueante. Durante esta fase, el sistema:

1. Congela la interacción del usuario mientras el proceso está en curso.
2. Valida el saldo disponible en tiempo real.
3. Debita el monto de la cuenta origen.
4. Acredita el monto en la cuenta destino.
5. Devuelve una confirmación definitiva al cliente.

Este modelo síncrono previene situaciones de fraude como el *double spending* (doble gasto de fondos), en que un mismo saldo podría utilizarse en dos transacciones concurrentes si no existiera el bloqueo atómico [8].

### Fase asíncrona: operaciones no bloqueantes

Una vez que la transacción principal ha sido confirmada y el cliente ha recibido su «OK», el sistema encola de forma asíncrona un conjunto de tareas secundarias:

- Envío del SMS o notificación push de alerta al cliente.
- Generación del archivo de recibo en formato PDF.
- Registro de la transacción en el sistema de auditoría.
- Análisis post-transaccional para detección de patrones de fraude.
- Actualización de reportes y estados de cuenta.

Todas estas tareas se procesan de forma diferida, sin afectar el tiempo de respuesta percibido por el usuario, y son gestionadas por workers asíncronos a través de colas de mensajes.

## Tecnologías utilizadas en la arquitectura bancaria

Table 2: Tecnologías en la arquitectura bancaria híbrida

| Operación                 | Modelo    | Tecnología típica        |
|---------------------------|-----------|--------------------------|
| Autenticación             | Síncrono  | REST API / gRPC          |
| Validación de saldo       | Síncrono  | REST API / HTTP Blocking |
| Transferencia de fondos   | Síncrono  | REST API / gRPC          |
| Envío de SMS/notificación | Asíncrono | Kafka / RabbitMQ         |
| Generación de recibo PDF  | Asíncrono | RabbitMQ / ActiveMQ      |
| Análisis antifraude       | Asíncrono | Apache Kafka             |

## Tecnologías de Comunicación en Detalle

### Tecnologías síncronas

#### *RPC (Remote Procedure Call)*

El *Remote Procedure Call* (RPC) es un mecanismo de comunicación distribuida que permite a un programa invocar procedimientos o funciones que residen en un nodo remoto de la red como si fueran llamadas locales, ocultando toda la complejidad del protocolo de red subyacente. El modelo RPC es síncrono por definición: el proceso llamador queda bloqueado hasta recibir el resultado del proceso remoto [23]. Tecnologías modernas como gRPC extienden este concepto incorporando serialización binaria eficiente (*Protocol Buffers*), multiplexación HTTP/2 y soporte para streaming bidireccional.

#### *REST (Representational State Transfer)*

REST es el estilo arquitectónico más utilizado para el diseño de APIs web. Se basa en los principios de statelessness (sin estado), identificación de recursos mediante URIs, manipulación de recursos a través de métodos HTTP estándar (GET, POST, PUT, DELETE) y representaciones transferibles en formatos como JSON o XML [5].

Su modelo de comunicación es síncrono: cada petición HTTP espera su respuesta correspondiente antes de que el cliente pueda continuar.

### *HTTP Blocking*

El modelo de comunicación HTTP por defecto (*HTTP/1.1*) implementa un mecanismo de bloqueo en el que cada petición ocupa un hilo de ejecución del servidor hasta que la respuesta es enviada. Bajo alta concurrencia, esto puede convertirse en un cuello de botella significativo. *HTTP/2* introduce la multiplexación de solicitudes sobre una única conexión TCP, lo que reduce parcialmente este problema, aunque el modelo de espera del cliente sigue siendo síncrono desde la perspectiva de la aplicación [7].

## **Tecnologías asíncronas**

### *Apache Kafka*

Apache Kafka es una plataforma de streaming distribuido diseñada para el procesamiento de eventos en tiempo real a alta escala. Su arquitectura se basa en *tópicos* particionados y replicados, que permiten el consumo paralelo de mensajes por múltiples grupos de consumidores simultáneamente. Kafka es especialmente adecuado para pipelines de datos, ingesta de logs, análisis en tiempo real y escenarios donde la durabilidad y el alto rendimiento son prioritarios [19].

### *RabbitMQ*

RabbitMQ es un broker de mensajes de propósito general que implementa el protocolo AMQP (*Advanced Message Queuing Protocol*). Ofrece enrutamiento flexible mediante intercambiadores (*exchanges*), soporte para múltiples patrones de mensajería (directo, fanout, topic, headers) y garantías configurables de entrega (at-most-once, at-least-once). Es la solución preferida cuando la prioridad es la fiabilidad de entrega y la flexibilidad de enrutamiento [20].

## **Discusión y Análisis Crítico**

---

### **Impacto en el rendimiento**

Investigaciones recientes en el campo de los sistemas distribuidos han demostrado empíricamente que la comunicación asíncrona basada en EDA produce mejoras sustanciales de rendimiento frente a la comunicación síncrona basada en APIs HTTP, especialmente bajo condiciones de alta carga concurrente [21]. Sin embargo, esta ventaja tiene un



costo: la mayor complejidad de implementación y depuración, la necesidad de gestionar la consistencia eventual y la dificultad para rastrear flujos de ejecución distribuidos.

### El dilema de la consistencia

Uno de los desafíos fundamentales de los sistemas distribuidos asíncronos es la **consistencia eventual**: dado que los mensajes se procesan de forma diferida, puede existir un período de tiempo en que diferentes nodos del sistema tengan visiones inconsistentes del estado global. Para aplicaciones financieras o de misión crítica, este comportamiento es inaceptable en las operaciones principales, razón por la cual se recurre al modelo híbrido analizado en el caso bancario [16].

### Escalabilidad y resiliencia

La arquitectura asíncrona basada en colas de mensajes y brokers permite escalar horizontalmente los consumidores de forma independiente al productor. Ante un pico de tráfico, los mensajes se acumulan en la cola y son procesados gradualmente, evitando la sobrecarga del sistema. Esta característica hace que las arquitecturas EDA sean especialmente adecuadas para plataformas con millones de usuarios concurrentes, como las mencionadas en el caso de Uber y TikTok [13].

### Tendencias actuales

Las tendencias emergentes en el diseño de sistemas distribuidos apuntan hacia la adopción de arquitecturas nativas en la nube que combinen los dos paradigmas de forma inteligente, utilizando tecnologías como Apache Kafka para el procesamiento de streams, gRPC para la comunicación de baja latencia entre microservicios, y plataformas de orquestación como Kubernetes para gestionar el ciclo de vida de los componentes [24].

## Conclusiones

---

A partir del análisis desarrollado a lo largo del presente informe, es posible establecer las siguientes conclusiones:

1. La **computación síncrona** y la **computación asíncrona** son paradigmas complementarios, no excluyentes. Los sistemas distribuidos modernos de mayor complejidad y escala adoptan arquitecturas híbridas que combinan ambos modelos según los requisitos específicos de cada operación.
2. El modelo **síncrono** es la opción natural cuando se requiere confirmación inmediata del resultado de una operación, especialmente en transacciones financieras, autenticación

o consultas de datos en tiempo real. Su principal limitación es el bloqueo del hilo de ejecución y la baja tolerancia a fallos.

3. El modelo **asíncrono** es superior en escenarios de alta concurrencia, operaciones de larga duración y sistemas que deben absorber picos de tráfico sin degradar la experiencia del usuario. Su complejidad de implementación y la necesidad de gestionar la consistencia eventual son los principales desafíos a considerar.
4. Las tecnologías de **message broker** —Apache Kafka, RabbitMQ, ActiveMQ— son el pilar de la comunicación asíncrona en arquitecturas de microservicios y sistemas distribuidos modernos, proporcionando persistencia, desacoplamiento y escalabilidad horizontal.
5. La **Arquitectura Basada en Eventos** (EDA) representa la evolución natural de los sistemas distribuidos asíncronos, ofreciendo mejoras medibles de rendimiento, escalabilidad y resiliencia respecto a las arquitecturas puramente síncronas basadas en APIs REST.
6. La industria bancaria ilustra de forma paradigmática cómo los sistemas de producción reales utilizan ambos paradigmas: el modelo síncrono garantiza la *atomicidad* e *integridad* de las operaciones financieras críticas, mientras que el modelo asíncrono optimiza las operaciones secundarias y mejora la experiencia del usuario.

## Referencias

---

## References

---

- [1] A. Rahmatulloh, F. Nugraha, R. Gunawan, I. Darmawan, E. Haerani y R. Rizal, “Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems,” *2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS)*, pp. 1–6, 2022. doi: [10.1109/ICADEIS56544.2022.10037390](https://doi.org/10.1109/ICADEIS56544.2022.10037390).
- [2] N. Surantha, O. K. Utomo, E. M. Lionel, I. D. Gozali y S. M. Isa, “Intelligent Sleep Monitoring System Based on Microservices and Event-Driven Architecture,” *IEEE Access*, vol. 10, pp. 42069–42085, 2022. doi: [10.1109/ACCESS.2022.3167637](https://doi.org/10.1109/ACCESS.2022.3167637).
- [3] B. Charron-Bost, F. Pedone y A. Schiper (Eds.), “Replication: Theory and Practice,” *Lecture Notes in Computer Science*, vol. 5959, Springer, Berlin, Heidelberg, 2010. doi: [10.1007/978-3-642-11294-2](https://doi.org/10.1007/978-3-642-11294-2).
- [4] R. van Glabbeek, U. Goltz y J.-W. Schicke, “On Synchronous and Asynchronous Interaction in Distributed Systems,” *Electronic Notes in Theoretical Computer Science*, vol. 209, pp. 167–197, 2008. doi: [10.1016/j.entcs.2008.04.011](https://doi.org/10.1016/j.entcs.2008.04.011).
- [5] G. Blinowski, A. Ojdowska y A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022. doi: [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803).
- [6] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Doctoral dissertation, University of California, Irvine, 2000. [En línea]. Disponible: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [7] S. Li et al., “Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review,” *Information and Software Technology*, vol. 131, p. 106449, 2021. doi: [10.1016/j.infsof.2020.106449](https://doi.org/10.1016/j.infsof.2020.106449).
- [8] Y. Abgaz et al., “Decomposition of Monolith Applications into Microservices Architectures: A Systematic Review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023. doi: [10.1109/TSE.2023.3287297](https://doi.org/10.1109/TSE.2023.3287297).
- [9] D. Weyns, “An Introduction to Multiagent Systems,” *2nd ed.*, Wiley, 2010. doi: [10.1145/1831690.1831731](https://doi.org/10.1145/1831690.1831731).
- [10] E. Laigner, M. Kalinowski, P. Diniz, L. Barros, C. Cassino, M. Lemos, D. Arruda, S. Lifschitz y Y. Zhou, “From a Monolithic Big Data System to a

- Microservices Event-Driven Architecture,” *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 213–220, 2020. doi: [10.1109/SEAA51224.2020.00042](https://doi.org/10.1109/SEAA51224.2020.00042).
- [11] A. Akbulut y H. G. Perros, “Performance Analysis of Microservice Design Patterns,” *IEEE Internet Computing*, vol. 23, no. 6, pp. 19–27, Nov.–Dic. 2019. doi: [10.1109/MIC.2019.2951337](https://doi.org/10.1109/MIC.2019.2951337).
- [12] T. Cerny, M. J. Donahoo y M. Trnka, “Contextual Understanding of Microservice Architecture: Current and Future Directions,” *ACM SIGAPP Applied Computing Review*, vol. 17, no. 4, pp. 29–45, 2017. doi: [10.1145/3183628.3183631](https://doi.org/10.1145/3183628.3183631).
- [13] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li y D. Ding, “Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study,” *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021. doi: [10.1109/TSE.2018.2887384](https://doi.org/10.1109/TSE.2018.2887384).
- [14] M. Fowler y J. Lewis, “Microservices: a definition of this new architectural term,” [martinfowler.com](https://martinfowler.com/articles/microservices.html), 2014. [En línea]. Disponible: <https://martinfowler.com/articles/microservices.html>.
- [15] A. Tanenbaum y M. van Steen, *Distributed Systems: Principles and Paradigms*, 3a ed. Prentice Hall, 2016.
- [16] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [17] I. Weber, X. Xu, R. Riveret, G. Governatori, A. Ponomarev y J. Mendling, “Untrusted Business Process Monitoring and Execution Using Blockchain,” *Lecture Notes in Computer Science*, vol. 9850, pp. 329–347, 2016. doi: [10.1007/978-3-319-45348-4\\_19](https://doi.org/10.1007/978-3-319-45348-4_19).
- [18] D. E. Bakken, “Middleware,” en *Encyclopedia of Distributed Computing*, Kluwer Academic Publishers, 2002.
- [19] J. Kreps, N. Narkhede y J. Rao, “Kafka: A Distributed Messaging System for Log Processing,” *Proc. of NetDB Workshop at VLDB*, 2011. [En línea]. Disponible: <http://sites.computer.org/debull/A12june/pipeline.pdf>.
- [20] A. Videla y J. J. W. Williams, *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, 2012.
- [21] S. Hassan, N. Ali y R. Bahsoon, “Microservice Ambients: An Architectural Meta-Modelling Approach for Microservice Granularity,” *2017 IEEE*

- International Conference on Software Architecture (ICSA)*, pp. 1–10, 2017. doi: [10.1109/ICSA.2017.32](https://doi.org/10.1109/ICSA.2017.32).
- [22] N. Dragoni et al., “Microservices: Yesterday, Today, and Tomorrow,” en *Present and Ulterior Software Engineering*, Springer, pp. 195–216, 2017. doi: [10.1007/978-3-319-67425-4\\_12](https://doi.org/10.1007/978-3-319-67425-4_12).
- [23] A. D. Birrell y B. J. Nelson, “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, Feb. 1984. doi: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392).
- [24] D. Taibi, V. Lenarduzzi y C. Pahl, “Architectural Patterns for Microservices: A Systematic Mapping Study,” *Proc. of the 8th International Conference on Cloud Computing and Services Science (CLOSER 2018)*, pp. 221–232, 2018. doi: [10.5220/0006798302210232](https://doi.org/10.5220/0006798302210232).